

Smart Guard — Part 4 explain

Contents

Part 4 — Code & Architecture Explanation	1
1. Goals (from the course brief)	1
2. System architecture (Part 4)	2
3. File map	2
4. Process startup	3
5. Feature 4-1 — Guard (anti-theft) mode	3
6. Feature 4-2 — Black box	5
7. Feature 4-3 — Software watchdog	6
8. Feature 4-4 — Adaptive thermal management	7
9. REST surface added/used in Part 4	9
10. Persistence helpers	9
11. Timing / rate limits (coordinator)	10
12. Failure modes & design rationale	10
13. Configuration keys (Part 4)	10
14. End-to-end scenarios	10
15. How Part 4 depends on earlier parts	11
16. Quick reference	11
17. PDF report package (what this part must contain)	11
18. Results analysis (Part 4)	12
19. Problems encountered and how they were solved (Part 4)	12
20. Conclusion	13
21. Related documents	13

Part 4 — Code & Architecture Explanation

Field	Value
Document	Final architecture & code explanation (submission)
Status	Complete — aligned with report/part 4/report.md
Companion	report/part 4/report.md (mandatory experiments 4-1 ... 4-4)
Also covers	PDF package: architecture, code, experiment videos/images, results analysis, problems & solutions
Project	Smart Guard System
Student	Parsa Nikookalam · 402102657
Scope	Guard, black box, software watchdog, adaptive thermal (C coordinator; enter ≥ 80 °C, clear < 78 °C)

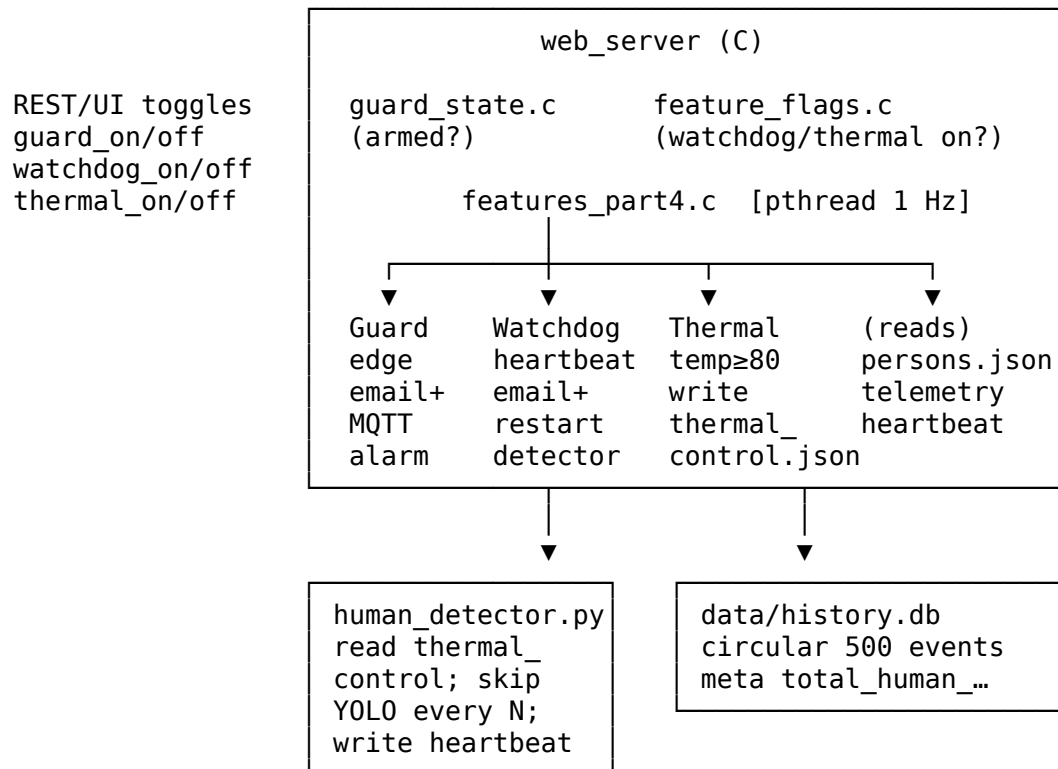
Part 4 is a **policy coordinator** on top of Parts 1-3. It does not replace the web server or detector; it applies Guard / watchdog / thermal / recording rules on the same shared files and APIs.

1. Goals (from the course brief)

#	Feature	Expected demo	Code home
4-1	Guard mode	Video/images of alarm behaviour	features_part4.c + guard_state.c
4-2	Black box	DB / API screenshot of stored events	Detector SQLite + persons_state.c +
4-3	Software watchdog	Disconnect camera → reaction video/image	Heartbeat file + watchdog branch in f
4-4	Adaptive thermal	Stress CPU → throttle images	Thermal thresholds + thermal_contro

Dashboard toggles: **Guard / Watchdog / Thermal / Camera.**

2. System architecture (Part 4)



Single coordinator thread (part4_thread) polls once per second. That keeps Guard edges responsive without busy-waiting inside the HTTPS accept loop.

3. File map

```

web/src/
├── features_part4.c/.h    # Coordinator thread (Guard + WD + thermal)
├── guard_state.c/.h      # Persist Guard armed flag
├── feature_flags.c/.h    # Persist watchdog + thermal enabled
├── camera_state.c/.h     # Camera on/off file for detector
├── email_alert.c         # email_send_event() used by Part 4
├── mqtt_pub.c            # mqtt_publish_alarm()
├── persons_state.c       # blackbox stats + persons snapshot
└── detection/src/human_detector.py
    ├── BLACKBOX_CAPACITY = 500
    └── write vision_heartbeat.json (ts on real frames only)
  
```

```

- read thermal_control.json (detect_every, yolo_input)
data/ (paths from C: ../data/... relative to WorkingDirectory=web/)
├── guard_state.json      # {"armed":0|1}          - default disarmed
├── watchdog_state.json   # {"enabled":0|1}       - default ON if file missing
├── thermal_state.json    # {"enabled":0|1}       - default ON if file missing
├── camera_state.json     # {"enabled":0|1}       - default OFF
├── vision_heartbeat.json # {"ts":..., "mode": "idle"|"capturing"|...}
├── thermal_control.json  # throttle_level, detect_every, yolo_input (C→Python)
├── persons.json
├── history.db
└── latest_detection.jpg

```

Defaults in code: feature_flags.c initializes watchdog/thermal to **enabled** when the state file is missing (return 1). Guard (guard_state.c) and camera (camera_state.c) default to **off**.

4. Process startup

From main.c:

```

email_alert_start();
mqtt_pub_start();
part4_start();          /* creates detached part4_thread */
start_http_server(0);

```

part4_start():

```

pthread_attr_setdetachstate(..., PTHREAD_CREATE_DETACHED);
pthread_create(..., part4_thread, NULL);

```

On entry, the thread loads thresholds from config.env and writes an initial thermal_control.json with throttle level 0.

4.1 Source: starting the coordinator

```

/* web/src/features_part4.c */
void part4_start(void)
{
    pthread_t th;
    pthread_attr_t attr;
    pthread_attr_init(&attr);
    pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
    if (pthread_create(&th, &attr, part4_thread, NULL) != 0)
        fprintf(stderr, "[part4] failed to start thread\n");
    pthread_attr_destroy(&attr);
}

```

The loop body runs Guard → Watchdog → Thermal once per second (sleep(1)).

5. Feature 4-1 — Guard (anti-theft) mode

5.1 State

Persisted in data/guard_state.json as {"armed":0|1} via guard_state.c (atomic write through .tmp + rename).

Default when missing: **disarmed**.

REST:

- POST {"cmd":"guard_on"} / guard_off (aliases guard_arm / guard_disarm also accepted)
- GET /api/v1/guard → {"armed":true|false}

5.2 Trigger condition (important)

```
if (guard_is_armed() && count > prev_count) {
    /* fire alarm */
}
prev_count = count;
```

Event	Guard fires?
0 → 1 persons	Yes
1 → 2 persons	Yes
1 stays 1	No (Part 3 presence email may still debounce)
2 → 1	No (decrease)

This matches “someone new appeared / count increased” anti-theft semantics better than “any time count $\geq$ 1”.

5.3 Source: Guard edge (features_part4.c)

```
/* ---- 4.1 Guard / anti-theft ----
 * Fast alarm on any INCREASE in person count (0→1, 1→2, ...).
 */
if (guard_is_armed() && count > prev_count) {
    if (now - last_guard_mail >= 2) {
        snprintf(body, sizeof(body),
            "GUARD ALARM (anti-theft)\nStudent ID: %s\n"
            "Persons: %d → %d (increase)\nCPU temp: %.2f C\n"
            "Timestamp: %ld\nMQTT topic: home/%s/alarm\n",
            g_student_id, prev_count, count, temp, now, g_student_id);
        email_send_event(subj, body, 1); /* attach photo */
        mqtt_publish_alarm(count, temp, now); /* home/<ID>/alarm */
        last_guard_mail = now;
    }
}
prev_count = count;
```

Alarm publish (mqtt_pub.c):

```
int mqtt_publish_alarm(int count, float cpu_temp, long timestamp)
{
    snprintf(topic, sizeof(topic), "home/%s/alarm", g_student);
    snprintf(payload, sizeof(payload),
        "{\"alarm\":true,\"count\":%d,\"cpu_temp\":%.2f,\"timestamp\":%ld}",
        count, cpu_temp, timestamp);
    return mosquitto_publish(g_mosq, NULL, topic, (int)strlen(payload), payload, 1, false);
}
```

Guard persistence (guard_state.c):

```
#define GUARD_PATH "../data/guard_state.json"

static void write_file_unlocked(int armed)
{
    fprintf(fp, "{\"armed\":%d}\n", armed ? 1 : 0);
}
```

```

    rename(GUARD_TMP, GUARD_PATH);
}

```

5.4 Actions on fire

1. Build text body with student ID, old→new count, CPU temp, timestamp.
2. `email_send_event(..., attach_photo=1)` — **fast** path (min gap ~2 s between Guard mails).
3. `mqtt_publish_alarm(count, temp, ts)` → topic:

```

home/<STUDENT_ID>/alarm
{"alarm":true,"count":N,"cpu_temp":T,"timestamp":TS}

```

4. Log [part4] GUARD alarm a → b.

5.4 Interaction with Part 3 email

Path	When	Debounce
Part 3 presence	count ≥ 1	~30 s
Part 4 Guard	count increases while armed	~2 s

Both can coexist: Guard is the urgent edge; Part 3 is periodic presence.

5.5 Demo checklist (code-aligned)

1. `guard_on + camera_on`.
2. `mosquitto_sub -t home/402102657/alarm`.
3. Walk into frame → expect mail + MQTT.
4. Capture video/screenshots for report experiment 4-1.

6. Feature 4-2 — Black box

6.1 Writer (Python)

On detection updates, `human_detector.py` calls `append_history(...)`:

- `INSERT INTO detections(count, timestamp)`
- If `bump_total` and `count > 0`: increment `meta.total_human_events`
- Prune to last **BLACKBOX_CAPACITY (500)** rows (`DELETE ... NOT IN (... LIMIT 500)`)

Database file: `data/history.db`.

6.2 Source: circular buffer writer (`human_detector.py`)

```

def append_history(count: int, ts: int, *, bump_total: bool = False) -> None:
    conn = sqlite3.connect(HISTORY_DB)
    conn.execute(
        "INSERT INTO detections (count, timestamp) VALUES (?, ?)",
        (int(count), int(ts)),
    )
    if bump_total and int(count) > 0:
        conn.execute(
            "UPDATE meta SET value = value + 1 WHERE key='total_human_events'"
        )
    conn.execute(f"""

```

```

        DELETE FROM detections WHERE id NOT IN (
            SELECT id FROM detections ORDER BY id DESC LIMIT {BLACKBOX_CAPACITY}
        )
    """
    conn.commit()

```

6.3 Reader (C)

persons_state.c:

Function	API
read_persons_history	GET /api/v1/history (last 5)
read_blackbox_stats	GET /api/v1/blackbox

GET /api/v1/blackbox response shape (from server.c):

```
{"total_human_events": 132, "stored": 47, "capacity": 500}
```

- total_human_events — from SQLite meta table (lifetime counter)
- stored — COUNT(*) of rows currently in detections
- capacity — BLACKBOX_CAPACITY (500)

Opens SQLite **read-only** so the API cannot corrupt the writer's DB.

6.3 Why “black box”?

Like an aviation recorder: even if the live stream is gone and count returns to 0, **past events remain** (until capacity eviction). Experiment 4-2 asks for a screenshot of those stored events (API JSON, UI, or sqlite3 SELECT).

6.4 Example inspection

```
curl -sk https://127.0.0.1:8443/api/v1/blackbox
```

```
sqlite3 data/history.db \
```

```
"SELECT id,count,datetime(timestamp,'unixepoch') FROM detections ORDER BY id DESC LIMIT 20;
```

7. Feature 4-3 — Software watchdog

7.1 Heartbeat contract

Detector writes data/vision_heartbeat.json:

```
{"ts": 1710000123, "mode": "capturing"}
```

Critical design rule: write_heartbeat(mode, touch_ts=...) in Python:

- touch_ts=True → update ts (healthy real frame, or idle with camera off).
- touch_ts=False → keep previous ts while mode stays non-idle (opening camera, waiting for usbipd, read failures).

Watchdog only evaluates when mode != "idle". Stalled camera paths therefore call write_heartbeat("capturing", touch_ts=False) so ts ages and the watchdog can fire. Successful frames use touch_ts=True.

mode	Meaning
idle	Camera OFF — watchdog does not treat this as tampering

mode	Meaning
capturing (non-idle)	Camera intended ON — ts must keep advancing on real frames

7.2 Source: watchdog branch (features_part4.c)

```

if (watchdog_is_enabled() && read_heartbeat(&hb_ts, mode, sizeof(mode)) == 0) {
    long age = (hb_ts > 0) ? (now - hb_ts) : 0;
    int watching = (strcmp(mode, "idle") != 0);
    if (watching && hb_ts > 0 && age > g_watchdog_sec && age < 3600) {
        if (now - last_watch_mail >= 45) {
            email_send_event("[Smart Guard] WARNING — camera tampering", body, 0);
            system("systemctl restart human_detector >/dev/null 2>&1");
            last_watch_mail = now;
        }
    }
}

```

Defaults: WATCHDOG_SEC=30 from config.env (minimum clamped to 10).

7.3 Enable flag

watchdog_on / watchdog_off via feature_flags.c → data/watchdog_state.json + GET /api/v1/watchdog.

Default: enabled if the file does not exist.

7.4 Disconnect-camera experiment (4-3)

1. Watchdog ON, Camera ON, healthy stream.
2. Unplug USB cam or usbipd detach.
3. After >30 s: tampering email + detector restart in journalctl.
4. Re-attach; detector reopens device (may need Camera toggle if capture handle is dead).

7.5 Why restart the detector?

A wedged OpenCV VideoCapture often will not recover in-process after USB loss. systemd restart gives a clean Python interpreter and re-init of ONNX nets + device nodes.

8. Feature 4-4 — Adaptive thermal management

8.1 Thresholds

From features_part4.c / config.env:

Name	Default (current config)	Meaning
THERMAL_TEMP_C	80	Enter / raise throttle
THERMAL_CLEAR_C	78	Hysteresis clear to level 0

Hysteresis avoids flapping around a single point.

8.2 Throttle levels written to JSON

`write_thermal_control(level, temp)` creates `data/thermal_control.json`:

level	yolo_input	detect_every	target_fps	Intent
0	640	1	0 (no override)	Full detect
1	640	2	12	YOLO every 2nd frame; soft FPS cap
2	416	4	5	Smaller input + stride 4; hard cap ~5 FPS

`frame_sleep_ms` is kept **0** on purpose: sleeping the capture loop made the MJPEG stream look frozen on WSL. Skipping inference / capping FPS keeps the pipeline live.

Level 2 is chosen when `temp >= THERMAL_TEMP_C + 8` (≈ 88 °C with the current 80 °C enter threshold).

8.3 Source: write control file + throttle decision (`features_part4.c`)

```
static void write_thermal_control(int level, float temp)
{
    int yolo = 640;
    int detect_every = 1;
    int target_fps = 0;
    if (level >= 2) {
        yolo = 416;
        detect_every = 4;
        target_fps = 5;
    } else if (level == 1) {
        yolo = 640;
        detect_every = 2;
        target_fps = 12;
    }
    fprintf(fp,
        "{ \"throttle_level\": %d, \"cpu_temp\": %.2f, \"yolo_input\": %d, \"detect_every\": %d, \"target_fps\": %d, \"frame_sleep_ms\": 0 } \n",
        level, temp, yolo, detect_every, target_fps);
}

/* in part4_thread: */
if (thermal_is_enabled() && temp > 0) {
    int want = throttle_level;
    if (temp >= g_thermal_on)
        want = (temp >= g_thermal_on + 8.0f) ? 2 : 1;
    else if (temp <= g_thermal_off)
        want = 0;
    if (want != throttle_level) {
        throttle_level = want;
        write_thermal_control(throttle_level, temp);
        if (throttle_level > 0)
            email_send_event("[Smart Guard] Thermal throttle active", body, 0);
    }
}
```

8.4 Detector side

Each loop iteration:

```
thermal = read_thermal_control()
detect_every = max(1, int(thermal["detect_every"]))
run_detect = (frame_i % detect_every) == 0
```

Overlay may show THR{level} plus stride / target_fps (e.g. level 2 → target_fps=5).

8.5 Email on enter

When throttle level becomes > 0, C sends a thermal notification (rate-limited ~60 s).

8.6 Enable flag

thermal_on / thermal_off → data/thermal_state.json + GET /api/v1/thermal.

Default: enabled if the file does not exist.

If disabled while throttling, coordinator forces level 0 and rewrites thermal_control.json.

8.7 Stress tools for experiment 4-4

```
stress-ng --cpu 0 --timeout 300s &
curl -sk https://127.0.0.1:8443/api/v1/telemetry
cat data/thermal_control.json
```

On WSL, ensure HostCpuTemp is updating so telemetry.c sees rising °C. For demos, temporarily lowering THERMAL_TEMP_C is acceptable if noted in the report.

9. REST surface added/used in Part 4

Method	Path	Role
GET	/api/v1/guard	Armed?
GET	/api/v1/watchdog	Enabled?
GET	/api/v1/thermal	Enabled?
GET	/api/v1/camera	Enabled?
GET	/api/v1/part4	Combined snapshot for UI
GET	/api/v1/blackbox	Recorder stats
POST	/api/v1/command	All *_on / *_off cmds

Command names are listed in the 400 hint / gateway OpenAPI examples.

10. Persistence helpers

10.1 guard_state.c

File: ../data/guard_state.json. Mutex-protected in-memory cache + disk write so guard_on survives process restart until explicitly disarmed.

10.2 feature_flags.c

Files: ../data/watchdog_state.json, ../data/thermal_state.json. Same pattern; missing file ⇒ feature **ON**.

10.3 camera_state.c

File: ../data/camera_state.json. Bridge from C command handler to Python. Detector also keeps an in-memory copy (and re-reads the file) so UI toggles apply without relying only on a stale cache.

11. Timing / rate limits (coordinator)

Concern	Interval
Coordinator loop	sleep(1)
Guard email min gap	2 s
Watchdog email min gap	~45 s
Thermal email min gap	~60 s
Rewrite thermal JSON while stable	~15 s
Reload Part 4 cfg from env	every ~60 ticks

These limits protect SMTP and journal spam during demos.

12. Failure modes & design rationale

Problem	Design choice
Fork-per-HTTP + MQTT threads	Use pthreads for HTTP + Part 4
Thermal sleep froze video	Skip YOLO frames instead
Heartbeat on idle fooled WD	Touch ts only on real frames
USB camera death	systemctl restart human_detector
Guard vs presence spam	Edge trigger + separate debounce

13. Configuration keys (Part 4)

```
THERMAL_TEMP_C=80
THERMAL_CLEAR_C=78
WATCHDOG_SEC=30
STUDENT_ID=402102657
```

Email/MQTT settings from Part 3 still required for Guard/watchdog/thermal notifications and ../alarm.

14. End-to-end scenarios

14.1 Guard

Armed → person enters → count↑ → email+photo + MQTT alarm within ~1-2 s poll.

14.2 Black box

Several detections → rows in history.db → /api/v1/blackbox shows totals/capacity.

14.3 Watchdog

Camera on + WD on → unplug cam → heartbeat stale → mail + restart → replug → stream returns.

14.4 Thermal

Thermal on → stress-ng → temp ≥ 80 → thermal_control.json level ≥ 1 → overlay THR / lower detect rate (level 2 → target_fps=5) → cool below 78 → level 0.

15. How Part 4 depends on earlier parts

Needs from...	What
Part 1	Always-on C process + systemd
Part 2	Telemetry temp, REST command bus, dashboard
Part 3	persons.json, snapshots, email/MQTT stacks, detector loop

Part 4 is policy + recording on top of that substrate.

16. Quick reference

```
curl -sk https://127.0.0.1:8443/api/v1/part4
```

```
curl -sk -X POST https://127.0.0.1:8443/api/v1/command \
-H 'Content-Type: application/json' -d '{"cmd":"guard_on"}'
curl -sk -X POST https://127.0.0.1:8443/api/v1/command \
-H 'Content-Type: application/json' -d '{"cmd":"watchdog_on"}'
curl -sk -X POST https://127.0.0.1:8443/api/v1/command \
-H 'Content-Type: application/json' -d '{"cmd":"thermal_on"}'
```

```
mosquitto_sub -h 127.0.0.1 -u smartguard -P 'smartguard' \
-t 'home/402102657/alarm' -v
```

```
journalctl -u web_server -u human_detector -f
```

17. PDF report package (what this part must contain)

Per the course PDF, Part 4 must include **architecture**, **code explanation**, **all experiment videos/images**, **results analysis**, and **problems & solutions**.

PDF expectation	Where it lives
Architecture	Sections 2–3 of this explain.md
Code explanation	Sections 4–12 (Guard, black box, watchdog, thermal) with source excerpts
Experiment media	report/part 4/report.md + fig/
Results analysis	Section 18 below
Problems & solutions	Section 19 below

17.1 Mandatory experiments — tables & figures checklist

No.	Experiment (PDF)	Required output	Files in fig/
4-1	Guard mode	Video + images of operation	01_guard_mode.mp4, 02_guard_er
4-2	Black box	Image of DB / API events	04_blackbox_events.png
4-3	Software watchdog	Disconnect camera; video + image of reaction	05_watchdog.mp4, 06_watchdog_
4-4	Adaptive thermal	Raise CPU; MQTT + email + lost FPS	07_thermal_mqtt.png, 08_therm

17.2 Example evidence tables

4-2 — Black box snapshot (typical measured run)

Field	Observed
total_human_events	132
stored	47 (≤ 500)
capacity	500

4-4 — Thermal phases

Phase	Temp	throttle_level	detect_every	target_fps	Observed FPS
Cool	< 78 °C	0	1	(vision default)	~12-18
Hot	≥ 80 °C	1	2	12	reduced / THR1
Very hot	$\geq \sim 88$ °C	2	4	5	~5 / THR2
Clear	falling < 78	0	1	(cleared)	recovers

18. Results analysis (Part 4)

Experiment	Expected outcome	Interpretat
4-1	Count increase → email + MQTT .../alarm while Guard armed	Edge trigg
4-2	Events remain in SQLite after count returns to 0	Circular bu
4-3	After >30 s without frames (camera disconnected) → tamper mail + detector restart	Heartbeat
4-4	Under stress heat, overlay shows THR / skip; stream stays live	Skipping Y

19. Problems encountered and how they were solved (Part 4)

#	Problem	Severity	Cause
1	Watchdog never emailed	Severe	Heartbeat ts updated even when frame
2	Thermal throttle froze MJPEG	Severe	Heavy sleep / tiny resolution in pipeline
3	Guard vs Part 3 email conflict confusion	Medium	Two email paths
4	Camera disconnect leaves dead OpenCV handle	High	USB/usbipd detach
5	Hard to reach 80+ °C on some demos	Medium	Cool laptop / WSL temp source
6	Black box vs history confusion	Low	Two APIs
7	Defaults: WD/thermal ON when files missing	Low (surprise)	feature_flags.c returns enabled if file

20. Conclusion

Part 4 implements four appliance behaviours in one C coordinator thread (features_part4.c):

Feature	Behaviour
Guard	Person-count increase → fast email + MQTT home/<ID>/alarm
Black box	Circular SQLite history (capacity 500); e.g. total_human_events=132, stored=47
Watchdog	Stale non-idle heartbeat → tamper email + systemctl restart human_detector
Thermal	≥80 °C throttle / <78 °C clear via thermal_control.json; level 2 → target_fps=5 ; stream stays

Experiments **4-1 ... 4-4** are complete with video/image/MQTT/email/FPS evidence under fig/. Together with Parts 1-3, this completes the Smart Guard System as specified in the course PDF.

21. Related documents

Document	Role
report/part 4/report.md	Mandatory experiments 4-1 ... 4-4 (final)
report/part 4/explain.md	This file — architecture & code (final)
report/part 3/explain.md	Vision / email / MQTT substrate
README.md (repo root)	Full-project setup guide